Mixed Majority-Minority Inverter Graphs for Inverter-Aware AQFP-Oriented Logic Optimization

Mrunal Shende

***Abstract*—In Adiabatic Quantum-Flux Parametron (AQFP) logic, every gate and inverter incurs measurable Josephson-junction cost, yet standard Majority-Inverter Graph (MIG) flows minimize gate count alone. We present an open-source mockturtle-based optimization flow for Mixed Majority-Minority Inverter Graphs (mMIGs) with CEC-guarded DSE and an AQFP-oriented inverter-aware objective: minority seeding, dual-inversion propagation, cone polarity flip, and CEC-guarded mixed refinement, reducing inverters by exploiting MAJ↔MIN duality. On twelve EPFL benchmarks, mMIG reduces inverter count by up to 34.8% and pre-mapping AQFP-proxy cost by up to 8.0% versus the paired pure-MIG seed (−12.4% and −4.7% in aggregate; no AQFP-proxy regression on any benchmark).**

***Index Terms*—logic synthesis, majority-inverter graph, AQFP, inverter minimization, design space exploration.**

I. Introduction

The Majority-Inverter Graph (MIG) is a directed acyclic graph in which every internal node computes the three-input majority function $M(a,b,c) = ab + bc + ca$, and edges carry optional complementation bits representing inverters [1]. Because inverters are "on-edge" and nominally free under standard cell assumptions, MIG optimization flows minimize gate count as the primary metric.

For *Adiabatic Quantum-Flux Parametron* (AQFP) superconducting logic, however, this assumption breaks down. Each splitter and inverter cell requires its own Josephson-junction (JJ) pair and an additional clocking zone, contributing directly to chip area and switching energy [2]. Optimizing for gate count alone can therefore miss inverter-related cost that is relevant in AQFP-oriented implementations.

In a pure-MIG representation, local rewrites have limited freedom to redistribute complemented edges because every internal node uses the same MAJ primitive. Allowing *minority* (MIN) gates alongside MAJ gates provides additional polarity choices: rewriting a cone of MAJ gates as MIN gates (and vice versa) enables local transformations that move boundary inversions across a cone — moves that are algebraically unavailable in a homogeneous MAJ network.

The algebraic foundation and early synthesis use of mixed MAJ/MIN networks were studied in prior mMIG work [3], [4]; the present work contributes, to the best of our knowledge, the first open-source MOCKTURTLE-based mMIG optimization flow with CEC-guarded DSE and an AQFP-oriented inverter-aware objective.

Contributions. This letter presents an open-source MOCKTURTLE-based mMIG optimization flow[1] (source available as a MOCKTURTLE [5] extension): minority seeding, dual-inversion propagation, cone polarity flip, an AQFP-oriented inverter-weighted cost objective, and a reproducible evaluation on twelve EPFL combinational benchmarks [6] against published MIG results [7].

II. mMIG: Definition and Cost Model

Definition 1 (mMIG [3], [4])**.** *A* Mixed Majority-Minority Inverter Graph *is a DAG $G = (V, E)$ where each internal node $v \in V$ is labelled either* MAJ *or* MIN*, has in-degree three, and each edge carries a complement attribute $c \in \{0, 1\}$. A complemented edge $(u, v, c{=}1)$ represents an implicit inverter between u and v. The function of a MAJ node is $M(a,b,c) = ab{+}bc{+}ca$; the function of a MIN node is $m(a,b,c) = \overline{M(a,b,c)}$.*

The central algebraic identities exploited throughout this work are:

$$\mathrm{MIN}(a,b,c) = \overline{\mathrm{MAJ}(a,b,c)} = \mathrm{MAJ}(\bar{a},\bar{b},\bar{c}), \tag{1}$$

$$\mathrm{MAJ}(\bar{a},\bar{b},c) = \mathrm{MIN}(a,b,\bar{c}). \tag{2}$$

Identity (1) shows that a MIN gate represents the opposite output polarity of a MAJ gate. When MIN is treated as an available primitive with the same gate cost as MAJ, it can absorb an output inversion that would otherwise appear as a complemented edge; the advantage of MIN as an explicit primitive is that it enables *topological* redistribution of inverters across subgraph boundaries — moves that have no direct analogue as native gate-type operations in a homogeneous MAJ network.

A. Inverter-Weighted Cost Objective

Standard MIG optimizers minimize gate count G alone, treating every complement edge as free. In our mMIG flow, the internal optimizer uses an *effective area score*:

$$\mathrm{score} = G \times 1024 \; + \; I_{\mathrm{eff}} \times w, \tag{3}$$

M. Shende is with the Department of Computer Science and Engineering, Indian Institute of Technology Indore, Madhya Pradesh, India 453552. E-mail: mrunalshende05@gmail.com.

[1]Source and reproducibility artifact: https://github.com/Mrunal321/mmig-esl-artifact

where I_{eff} is the *effective inverter count* and w is an adaptive weight. The effective inverter count credits each MIN gate with one absorbed inverter:

$$I_{\text{eff}} = I - \min(\text{MIN\_count},\ I), \tag{4}$$

since a MIN node represents the complemented output polarity of the corresponding MAJ node, so the downstream complement edge it would require is eliminated. I_{eff} serves as an internal search heuristic only; all reported results use the actual edge count I and proxy $A = 6G + 2I$. The weight $w \in \{1, 2, 4\}$ is set by the I/G density ratio: $w = 4$ for $I/G > 0.8$, $w = 2$ for $0.4 < I/G \leq 0.8$, else $w = 1$; scaling G by 1024 keeps the optimizer gate-dominant. At the outer DSE level, final candidates are ranked by $G{+}0.5I$ in area mode (Section IV-A), the selection objective for Table I. The proxy-consistent objective would be $G{+}\frac{1}{3}I$; $G{+}0.5I$ gives additional inverter-removal pressure because AQFP splitter trees and path-balancing buffers amplify inverter cost beyond the pre-mapping proxy. All reported results use $A = 6G + 2I$ regardless of the selection objective.

III. mMIG-Specific Optimization Passes

A. Minority Seeding

The optimizer first creates explicit MIN opportunities through *minority seeding*. For every MAJ node, it estimates the local inverter saving obtained by replacing the node with either $\overline{\text{MIN}}(a, b, c)$ or $\text{MIN}(\bar{a}, \bar{b}, \bar{c})$, ranks candidates by level-1 and level-2 inverter gain, and applies only candidates accepted by the configured equivalence guard. This step does not claim global optimality; it exposes MIN nodes that later passes can exploit to absorb complemented edges.

B. Dual-Inversion Propagation

We propose a *dual-inversion propagation* pass exploiting identities (1)–(2). *Triple inversion* (k=3 complemented inputs): by identity (1), $\text{MAJ}(\bar{a}, \bar{b}, \bar{c}) = \text{MIN}(a, b, c)$, so the pass flips the gate type and removes all three input complement edges with no output inversion required, giving a net saving of +3 (symmetric dual applies to MIN nodes). *Dual inversion* (k=2): by identity (2), $\text{MAJ}(\bar{a}, \bar{b}, c) = \text{MIN}(a, b, \bar{c})$, so the pass flips the gate type, moves the polarity to the remaining input, and toggles all fanout complement bits (one output polarity push). With fanout f and f_c complemented fanout edges, the net saving is $1{+}2f_c{-}f$; the rule is applied only when $f_c{>}(f{-}1)/2$ (Algorithm 2, lines 3–6).

C. Cone Polarity Flip

We introduce a *cone polarity flip* pass. For a connected subgraph (*cone*), the pass changes every node type (MAJ↔MIN) and updates edge complement attributes by endpoint membership. Edges with both endpoints inside the cone receive two complement toggles that cancel, leaving internal signal polarities unchanged. Only boundary edges (cone→outside) receive a single toggle; these absorb the overall sign change imposed by negating the cone output, so that the function seen outside the cone is preserved.

Algorithm 1 mMIG Design-Space Exploration

```
1: Input: circuit N; restarts R; steps S; bail-out K
2: Score: σ(N) = G + 0.5 I                                   (lower is better)
3: N* ← MIG-Seed(N)
4: for r = 1 to R do
5:     N_cur ← N*;   stall ← 0
6:     for t = 1 to S do
7:         N_d ← Decompress(N_cur)
8:         N_m ← mMIG-Opt(N_d)                                ▷ Alg. 2
9:         N_p ← MIG-Compress(N_d)
10:        c ← arg min{σ(N_m), σ(N_p)}
11:        if σ(c) < σ(N_cur) then N_cur ← c; stall ← 0
12:        else stall += 1; if stall ≥ K then break
13:        end if
14:    end for
15:    if σ(N_cur) < σ(N*) then N* ← N_cur
16:    end if
17: end for
18: return N*
```

Algorithm 2 mMIG-Opt: inverter-aware mMIG passes

```
1: Input: network N; inverter weight w
2: N' ← MinoritySeeding(N)          ▷ MAJ→MIN where ΔI > 0
3: for each v ∈ N' (topological order) do
4:     k ← |{complemented inputs of v}|
5:     if k = 3 then flip type(v); strip 3 input ¬ ▷ MAJ→MIN, no
       output ¬
6:     else if k = 2 and ΔI(v) > 0 then flip type(v); toggle
       remaining input ¬ and all fanout ¬ bits
7:     end if
8: end for
9: repeat                                              ▷ cone polarity flip
10:    for each cone C, |C| ≤ 8 do
11:        if |B⁻(C)| > |B⁺(C)| then FlipCone(C, N')
12:        end if
13:    end for
14: until no change
15: N' ← MIG-Engines(N', G·1024 + I_eff·w)
16: return N' if CEC(N, N') passes, else return N
```

Definition 2 (Boundary inverter delta)**.** *For a cone C, let B^+ be the set of non-complemented boundary edges and B^- the set of complemented boundary edges. The* boundary inverter delta *is $\delta = |B^+| - |B^-|$. Flipping C saves $-\delta$ inverters.*

Cones are grown by BFS (limit: 8 gates) and flipped only when $\delta \leq -1$; sweeps repeat until convergence (Algorithm 2, lines 9–14). This pass has no direct analogue as a native gate-type operation in a homogeneous MAJ network.

D. CEC-Guarded Mixed Refinement

After seeding and inverter propagation, the advanced optimizer applies the existing MIG rewriting, refactoring, resubstitution, exact rewriting, and balancing engines to the mixed network under the inverter-weighted objective. Because mixed MAJ/MIN transformations can change edge polarities in ways that are easy to mishandle, selected candidates are checked with a miter-based combinational equivalence guard before they are retained. This keeps the implementation compatible with the existing MIG engines while allowing the DSE to prefer solutions that reduce the explicit inverter count.

IV. Experimental Evaluation

A. Setup

All passes are implemented in the mockturtle C++ logic synthesis library [5]. We evaluate on 12 EPFL combinational benchmarks [6] using an on-the-fly design space exploration (DSE) framework [7] with 4 restarts and 20 steps per restart. Each step applies a randomly chosen ABC decompressor followed by either a pure-MIG compressor (*pure-MIG baseline*) or our mMIG compressor (*mMIG*). In area mode, the outer DSE ranks candidates by $G + 0.5I$, with depth and inverter count as tie-breakers.

We report gate count (G), complemented-edge count (I), and the pre-mapping AQFP proxy $A = 6G+2I$ [2] (6 JJ per MAJ/MIN gate, 2 JJ per inverter), which excludes fanout splitter trees and path-balancing buffers added during AQFP physical mapping. For context, Table I includes gate counts from three prior pure-MIG flows (Resyn. [8], DC-Rw. [9], MIG-DSE [7]); none reports AQFP-proxy cost, and Resyn./DC-Rw. omit depth. The MIG-DSE values are a published reference, not a controlled ablation; their search budget differs from ours. The controlled comparison is the paired pure-MIG seed versus mMIG from the *same* DSE row — both share the same decompressor step, isolating the mMIG contribution. Prior AQFP flows [10] use different benchmarks and include post-mapping buffer-insertion cost, so their numbers are not comparable with our pre-mapping proxy.

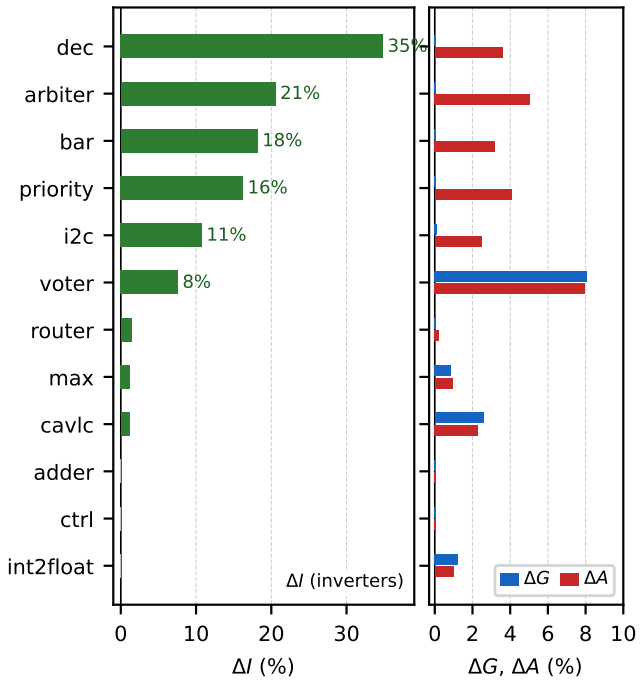

Fig. 1. Cost reduction of mMIG vs. paired pure-MIG baseline for all 12 EPFL benchmarks, sorted by ΔI (most improvement at top). *Left:* inverter savings (green). *Right:* gate (blue) and AQFP-proxy (red) savings. Positive values indicate improvement.

B. Results

Table I presents the full comparison including inverter count (I), depth (D), and the pre-mapping AQFP proxy for both baselines. Fig. 1 plots ΔG, ΔI, and ΔA per benchmark.

mMIG matches or improves the paired pure-MIG seed in AQFP-proxy cost on all 12 benchmarks. Inverter reduction is the primary driver of AQFP-proxy savings. The ΔI column shows that mMIG reduces inverter count on 9 of 12 benchmarks — up to 34.8% on `dec` (66→43) — with an overall total of −12.4% (13179→11547 inverters). Because each saved inverter eliminates 2 JJ, even modest ΔI translates directly to a lower pre-mapping JJ proxy: `arbiter` saves 914 inverters, which alone accounts for 1828 JJ of its 1834 JJ total saving. Gate count falls on 6 of 12 benchmarks and never increases, with the largest reduction at `voter`: −8.1% (5035→4629 gates). The AQFP proxy improves on 10 of 12 benchmarks and is unchanged on the remaining two, with a maximum saving of 8.0% (`voter`: 37138→34176 JJ). Totals across all 12: gates −2.6% (16727→16285), inverters −12.4% (13179→11547), AQFP proxy −4.7% (126720→120804 JJ). Circuit depth is maintained or reduced on 10 of 12 benchmarks; `cavlc` and `max` increase because the DSE prioritizes the inverter-weighted objective in area mode.

Comparison with MIG-DSE. MIG-DSE is a published reference point, not a controlled ablation. Our pure-MIG seeds are higher than MIG-DSE on most benchmarks for three reasons: (i) MIG-DSE calls SAT-based exact rewriting at each step, which finds reductions that our heuristic compressor misses; (ii) MIG-DSE runs a substantially larger search budget (more steps and restarts) tuned specifically for MIG gate minimization; and (iii) our DSE objective is $G+0.5I$, trading some gate reduction for inverter savings. mMIG gate count is still lower than MIG-DSE on `ctrl` (57 vs. 60) and `dec` (191 vs. 304). Since MIG-DSE reports no I or A for any benchmark, a direct AQFP-proxy comparison is not possible; the primary evidence of mMIG's contribution is the paired pure-MIG-to-mMIG reduction, which isolates the mMIG stage from all decompressor and search-budget effects. All outputs are verified equivalent via miter-based checking.

Comparison with Resyn. and DC-Rw. mMIG gate counts are no higher than Resyn. [8] and DC-Rw. [9] on 11 of 12 benchmarks each (`voter` exceeds both by 1.6%). On `arbiter` and `dec`, the pure-MIG seed gate count is already lower than both prior flows; the mMIG-specific contribution is isolated in the paired seed-to-mMIG columns.

Minority node statistics. `Arbiter` retains the most MIN nodes (597), followed by `priority` (80), `i2c` (72), and others; large MIN populations correlate with large ΔI. Notably, `voter` ends with zero MIN nodes yet still reduces gates and AQFP proxy, showing that the mixed representation also enables intermediate rewrites that return to a pure-MIG surface.

TABLE I

RESULTS ON 12 EPFL COMBINATIONAL BENCHMARKS [6]. RESYN. = HEURISTIC MIG RESYNTHESIS [8]; DC-RW. = MIG REWRITING WITH BOOLEAN DON'T CARES [9]; MIG-DSE = PUBLISHED MIG DESIGN-SPACE EXPLORATION GATE COUNTS [7]. GATE COUNT (G) ONLY FOR PRIOR WORKS; AQFP-PROXY COST NOT REPORTED BY ANY PRIOR WORK; DEPTH NOT REPORTED BY RESYN. OR DC-RW. PURE-MIG (OURS) = PAIRED PURE-MIG SEED BEFORE THE MMIG STAGE. MMIG (OURS) = THIS WORK. G = GATES; D = LOGIC DEPTH (BEFORE AQFP PATH BALANCING); I = INVERTER COUNT; A=6G+2I = PRE-MAPPING AQFP-ORIENTED JJ PROXY [2]. Δ = MMIG VS. PURE-MIG (%); ΔG_R AND ΔG_D COMPARE MMIG GATE COUNT WITH RESYN. AND DC-RW., RESPECTIVELY. NEGATIVE VALUES INDICATE LOWER COST FOR MMIG.

	Prior MIG flows (G only)			Pure-MIG (ours)				mMIG (ours)				vs pure-MIG (%)				vs prior (ΔG)	
Bench.	Resyn. [8]	DC-Rw. [9]	MIG-DSE [7]	G	D	I	A	G	D	I	A	ΔG	ΔD	ΔI	ΔA	ΔG_R	ΔG_D
adder	384	384	384	384	129	256	2816	384	129	256	2816	0.0	0.0	0.0	0.0	0.0	0.0
arbiter	6719	6711	792	4558	31	4442	36232	4557	31	3528	34398	0.0	0.0	−20.6	−5.1	−32.2	−32.1
bar	2588	2433	1906	2286	16	1462	16640	2286	16	1196	16108	0.0	0.0	−18.2	−3.2	−11.7	−6.0
cavlc	533	492	374	504	15	357	3738	491	18	353	3652	−2.6	+20.0	−1.1	−2.3	−7.9	−0.2
ctrl	79	74	60	57	6	39	420	57	6	39	420	0.0	0.0	0.0	0.0	−27.8	−23.0
dec	304	304	304	191	4	66	1278	191	4	43	1232	0.0	0.0	−34.8	−3.6	−37.2	−37.2
i2c	932	871	636	871	14	742	6710	870	14	662	6544	−0.1	0.0	−10.8	−2.5	−6.7	−0.1
int2float	181	172	115	161	13	121	1208	159	12	121	1196	−1.2	−7.7	0.0	−1.0	−12.2	−7.6
max	2210	2190	1939	2186	127	1797	16710	2167	129	1775	16552	−0.9	+1.6	−1.2	−0.9	−1.9	−1.1
priority	431	406	337	363	41	363	2904	363	41	304	2786	0.0	0.0	−16.3	−4.1	−15.8	−10.6
router	151	147	97	131	13	70	926	131	13	69	924	0.0	0.0	−1.4	−0.2	−13.2	−10.9
voter	4561	4555	3894	5035	54	3464	37138	4629	51	3201	34176	−8.1	−5.6	−7.6	−8.0	+1.5	+1.6
Total	–	–	–	16727	–	13179	126720	16285	–	11547	120804	−2.6	–	−12.4	−4.7	–	–

Circuit depth. Depth is maintained or reduced on 10 of 12 benchmarks. The two increases (`cavlc` 15→18, `max` 127→129) reflect the DSE trading depth for a lower inverter-weighted score in area mode.

V. Related Work

MIG optimization has been studied extensively since Amarù *et al.* [1] introduced the MIG formalism. Algebraic rewriting [1] applies a set of commutativity, associativity, and majority axioms to locally reduce MIG size. Boolean techniques — cut rewriting, resubstitution, and refactoring — adapted from AIG methodology, provide complementary reductions [8], [9]. Graph remapping [11] converts a MIG into a k-LUT network and back, often escaping local optima. For AQFP synthesis, Testa *et al.* [10] apply a combined algebraic-Boolean flow with full post-mapping buffer-insertion cost — a different benchmark set and cost model from the pre-mapping proxy used here.

Lee *et al.* [7] propose an on-the-fly DSE that randomly sequences the above algorithms, providing a strong published reference point for EPFL MIG gate counts.

Combining majority and minority gates was introduced by Aalam *et al.* [3]; Shende and Mazumdar [4] derived a fuller axiomatic basis. To the best of our knowledge, the present work contributes the first open-source MOCKTURTLE-based mMIG implementation with CEC-guarded DSE and an AQFP-oriented inverter-aware optimization flow, enabling cone-flip and inverter-absorption moves that are not available as native gate-type operations in pure-MAJ flows.

VI. Conclusion

We presented mMIG, a mixed MAJ/MIN inverter graph representation with an executable optimization flow (minority seeding, dual-inversion propagation, cone polarity flip, CEC-guarded refinement). On twelve EPFL benchmarks, mMIG reduces inverter count by up to 34.8% and AQFP proxy cost by up to 8.0% versus the paired pure-MIG seed, with aggregate totals of −12.4% and −4.7% and no AQFP-proxy regression on any benchmark. Future work will add polarity-aware MIN divisor enumeration and a direct mMIG-to-AQFP mapper accounting for splitter trees, path-balancing buffers, and clock-zone constraints.

References

[1] L. Amarù, P.-E. Gaillardon, and G. De Micheli, "Majority-inverter graph: A new paradigm for logic optimization," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 5, pp. 806–819, 2016.

[2] N. Takeuchi, D. Ozawa, Y. Yamanashi, and N. Yoshikawa, "An adiabatic quantum flux parametron as an ultra-low-power logic device," in *Superconductor Science and Technology*, vol. 26, no. 3, 2013, p. 035010.

[3] U. Aalam, B. Mazumdar, and N. Hubballi, "mMIG: Inversion optimization in majority inverter graph with minority operations," *Integration*, vol. 81, pp. 195–210, 2021.

[4] M. Shende and B. Mazumdar, "Towards improving performance metrics of minority majority inverter graph (mMIG) circuits," in *Proceedings of IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2025.

[5] M. Soeken, H. Riener, W. Haaswijk *et al.*, "The EPFL logic synthesis libraries," *arXiv preprint arXiv:1805.05121*, 2022.

[6] L. Amarù, P.-E. Gaillardon, and G. De Micheli, "The EPFL combinational benchmark suite," in *Proceedings of International Workshop on Logic Synthesis (IWLS)*, 2015.

[7] S.-Y. Lee, A. Tempia Calvino, H. Riener, and G. De Micheli, "Late breaking results: Majority-Inverter graph minimization by design space exploration," in *Proceedings of the Design Automation Conference (DAC)*, 2024, pp. 1–2.

[8] S.-Y. Lee and G. De Micheli, "Heuristic logic resynthesis algorithms at the core of peephole optimization," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.

[9] A. Tempia Calvino and G. De Micheli, "Scalable logic rewriting using don't cares," in *Proceedings of DATE*, 2024.

[10] E. Testa, S.-Y. Lee, H. Riener, and G. De Micheli, "Algebraic and Boolean optimization methods for AQFP superconducting circuits," in *Proceedings of ASP-DAC*, 2021, pp. 779–785.

[11] A. Tempia Calvino, H. Riener, S. Rai, A. Kumar, and G. De Micheli, "A versatile mapping approach for technology mapping and graph optimization," in *Proceedings of ASP-DAC*, 2022, pp. 410–416.